

Interview Protocol and Design Rationale

PA2534 Master’s Thesis in Software Engineering

Decision-Making Under Uncertainty in Bug Triage and Defect Prioritisation

Naga Sai Abhiram Gopal Vemana Vineeth Kumar Vajja

Supervisor: Muhammad Laiq

May 2026

1 Methodological and Theoretical Design Rationale

Research Context: Human Decision-Making vs. Automation. Much of the existing literature on bug triage focuses on automated approaches, such as machine learning classifiers and text mining [1, 2]. However, empirical research shows that different automated models often disagree and struggle when data is noisy or incomplete [3]. In real software development, incoming bug reports are frequently vague, lack clear reproduction steps, or describe hard-to-reproduce issues [4–6]. When automation falls short, bug triage relies on human judgment, team discussion, and practical problem-solving [7, 8]. To understand how engineers make triage decisions under these uncertain conditions, this study draws on Naturalistic Decision Making (NDM) [9], Gary Klein’s Recognition-Primed Decision (RPD) model [10], and cognitive bias research in software engineering [11].

Instrument Structure and Flow. The interview guide contains 12 main questions divided into five thematic sections, following established qualitative guidelines in software engineering [12, 13]. The guide follows a funnel design: it begins with general background and baseline workflows (Sections 2.1–2.2), moves into specific strategies for handling uncertain bug reports (Section 2.3), and addresses organizational constraints like deadlines and workload (Section 2.4) once rapport is established. The interview ends with an open reflective section (Section 2.5).

Question Phrasing and Bias Control. Questions are structured following the recommendations of Hove and Anda [12]. We use open-ended “what” and “how” questions, avoiding “why” prompts that can make participants feel defensive and avoiding leading yes/no questions that limit detail [12].

Focus on Specific Recent Incidents. To prevent participants from giving abstract or idealized answers, questions in Sections 2.3 and 2.4 ask participants to describe specific, recent incidents from their own experience [14]. This approach grounds the findings in real operational practices rather than general opinions.

Research Rigor and Saturation. The study follows the qualitative trustworthiness criteria (credibility, transferability, dependability, and confirmability) defined by Lincoln and Guba [15, 16]. In line with saturation guidelines from Guest et al. [17], we aim for codebook stability within a focused sample of experienced software practitioners. All interview procedures, participant roles, and recording methods are documented for reporting in the methodology chapter [12, 13].

2 Interview Protocol

2.1 Participant Background and Professional Context

Purpose: Gathers demographic and organizational context to support qualitative transferability [13,15]. Understanding an engineer’s role, experience, and project tools (e.g., Jira, Bugzilla) provides the context needed to interpret how they make decisions during triage [8,10,18].

Q1 *Could you briefly describe your current role and roughly how long you have been involved in bug triage or defect prioritisation in your work?*

Q2 *What does your team and project context look like?*

Probes (cover all, if not already mentioned):

- Team size
- Type of software you build
- Issue-tracking tools you use (e.g., Jira, Bugzilla, GitHub Issues)

2.2 The General Triage Process

Purpose: Establishes the participant’s normal triage workflow before asking about difficult or uncertain situations. In practice, bug triage is a collaborative team activity [1,7]; understanding the baseline process helps the researcher see how missing information disrupts routine work [10].

Q3 *Could you walk me through what a typical bug triage session looks like for you, from the moment a new report arrives to when a decision is made about it?*

2.3 Uncertainty in Triage: Strategies and Heuristics (RQ1)

Purpose: Directly addresses **Research Objective 1 (O1)**, **Research Objective 2 (O2)**, and **Research Question 1 (RQ1)**. Incomplete descriptions and non-reproducible bugs are common sources of uncertainty in daily maintenance [4–6]. This section investigates: (1) how engineers recall and describe an uncertain bug report (Q4), (2) whether they first look for more information or make assumptions based on past experience (Q5) [9], (3) how they resolve unclear priorities using team discussion or reporter reputation (Q6) [8,19], and (4) the practical rules of thumb they have developed over time (Q7) [10,11,14].

Q4 *Can you think of a specific, recent situation where you received a bug report that was incomplete, ambiguous, or impossible to reproduce?*

Probes:

- What happened?
- What information was missing?
- What did you do next?

Q5 *When a bug report lacks clear reproduction steps or environmental details, what do you typically do first?*

Probe: Could you walk me through the last time that happened?

Q6 *How do you decide what priority to assign to a bug when its severity is unclear or contested?*

Probes:

- Could you describe a specific bug where you had to make that call?

- Does it change anything if the bug also can't be reproduced?

Q7 *Over time, what rules of thumb or mental shortcuts have you developed that you rely on when information in a bug report is missing or unclear?*

Probe (only if no example given spontaneously): Can you give an example of a recent case where you used one of these?

2.4 Organisational and Contextual Factors (RQ2)

Purpose: Directly addresses **Research Objective 3 (O3)** and **Research Question 2 (RQ2)**. Triage decisions are shaped by organizational constraints like deadlines, workload, and business goals [20, 21]. This section explores: (1) how upcoming release deadlines change the way uncertain bugs are handled (Q8) [5, 21], (2) how management expectations and business priorities influence bug priority (Q9) [7, 8, 19], and (3) how heavy workloads limit investigation time before a decision has to be made (Q10) [18, 22].

Q8 *Can you describe a specific situation where an upcoming release deadline or workload pressure changed how you handled a bug report that you were uncertain about?*

Q9 *How do management expectations or business priorities factor into your triage decisions when you are uncertain about a bug's severity or its reproducibility?*

Probe: Could you recall a specific instance where that happened?

Q10 *How does your current workload typically affect the amount of time you can spend investigating a bug before making a decision?*

Probe: Can you describe a specific recent case where your workload shaped that decision?

2.5 Reflection, Tooling, and Emergent Close

Purpose: Directly supports **Research Objective 4 (O4)**. Gathers practitioner suggestions for improving tools, workflows, or AI assistance for uncertain reports (Q11) [11, 18, 23]. An open closing question (Q12) ensures that any unexpected or important insights not covered earlier can be shared by the participant [15, 24].

Q11 *Based on your experience, what do you think would most improve the process of handling uncertain or incomplete bug reports, whether in terms of tooling, process, or team practice?*

Q12 *Is there anything else about how you make bug triage decisions under uncertainty, something important to you that we have not had a chance to discuss?*

References

- [1] J. Anvik and G. C. Murphy, "Reducing the effort of bug report triage: Recommenders for development-oriented decisions," *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 20, no. 3, pp. 1–35, 2011.
- [2] S. Mani, A. Sankaran, and R. Aralikkatte, "Deeptriage: Exploring the effectiveness of deep learning for bug triaging," in *Proceedings of the ACM India Joint International Conference on Data Science and Management of Data (CODS-COMAD '19)*, pp. 171–179, ACM, 2019.
- [3] D. Bowes, T. Hall, and J. Petrić, "Software defect prediction: do different classifiers find the same defects?," *Software Quality Journal*, vol. 26, no. 2, pp. 525–552, 2018.

- [4] N. Bettenburg, S. Just, A. Schröter, C. Weiss, R. Premraj, and T. Zimmermann, “What makes a good bug report?,” in *Proceedings of the 16th ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE ’08)*, pp. 308–318, ACM, 2008.
- [5] M. E. Joorabchi, M. Mirzaaghaei, and A. Mesbah, “Works for me! characterizing non-reproducible bug reports,” in *Proceedings of the 11th Working Conference on Mining Software Repositories (MSR ’14)*, pp. 62–71, ACM, 2014.
- [6] M. M. Rahman, F. Khomh, and M. Castelluccio, “Works for me! cannot reproduce – a large scale empirical study of non-reproducible bugs,” *Empirical Software Engineering*, vol. 27, no. 5, p. 111, 2022.
- [7] M. Begum and Y. Dittrich, “What happens in triage? an empirical study of bug triage in software product evolution,” Tech. Rep. TR-2024-210, IT University of Copenhagen, 2024.
- [8] D. Bertram, A. Volda, S. Greenberg, and R. Walker, “Communication, collaboration, and bugs: The social nature of issue tracking in small, colocated teams,” in *Proceedings of the 2010 ACM Conference on Computer Supported Cooperative Work (CSCW ’10)*, pp. 291–300, ACM, 2010.
- [9] R. Lipshitz, G. Klein, J. Orasanu, and E. Salas, “Taking stock of naturalistic decision making,” *Journal of Behavioral Decision Making*, vol. 14, no. 5, pp. 331–352, 2001.
- [10] G. A. Klein, *Sources of Power: How People Make Decisions*. MIT Press, 20th anniversary ed. ed., 2017.
- [11] R. Mohanani, I. Salman, B. Turhan, P. Rodríguez, and P. Ralph, “Cognitive biases in software engineering: A systematic mapping study,” *IEEE Transactions on Software Engineering*, vol. 46, no. 12, pp. 1318–1339, 2020.
- [12] S. Hove and B. Anda, “Experiences from conducting semi-structured interviews in empirical software engineering research,” in *11th IEEE International Software Metrics Symposium (METRICS ’05)*, pp. 10–23, IEEE, 2005.
- [13] P. Runeson and M. Höst, “Guidelines for conducting and reporting case study research in software engineering,” *Empirical Software Engineering*, vol. 14, no. 2, pp. 131–164, 2009.
- [14] K. A. Ericsson and H. A. Simon, *Protocol Analysis: Verbal Reports as Data*. MIT Press, revised ed., 1993.
- [15] Y. S. Lincoln and E. G. Guba, *Naturalistic Inquiry*. SAGE Publications, 1985.
- [16] A. Bryman, *Social Research Methods*. Oxford University Press, 5th ed., 2016.
- [17] G. Guest, A. Bunce, and L. Johnson, “How many interviews are enough? experimenting with data saturation and variability,” *Field Methods*, vol. 18, no. 1, pp. 59–82, 2006.
- [18] O. Baysal, R. Holmes, and M. W. Godfrey, “No issue left behind: Reducing information overload in issue tracking,” in *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE ’14)*, pp. 666–677, ACM, 2014.
- [19] P. J. Guo, T. Zimmermann, N. Nagappan, and B. Murphy, “Characterizing and predicting which bugs get fixed: An empirical study of microsoft windows,” in *Proceedings of the 32nd ACM/IEEE International Conference on Software Engineering (ICSE ’10)*, pp. 495–504, ACM, 2010.
- [20] L. Erlikh, “Leveraging legacy system dollars for e-business,” *IT Professional*, vol. 2, no. 3, pp. 17–23, 2000.

- [21] M. Kuutila, M. M"antyl"a, U. Farooq, and M. Claes, "Time pressure in software engineering: A systematic review," *Information and Software Technology*, vol. 121, p. 106257, 2020.
- [22] H. Jahanshahi, M. Cevik, K. Mousavi, and A. Bařar, "ADPTriage: Approximate dynamic programming for bug triage under uncertainty," *IEEE Transactions on Software Engineering*, vol. 49, no. 10, pp. 4594–4609, 2023.
- [23] U. B. Torun, M. T. Demircan, M. F. G"on, and E. T"uz"un, "Past, present, and future of bug tracking in the generative ai era," *ACM Transactions on Software Engineering and Methodology (TOSEM)*, 2026. Just Accepted.
- [24] V. Braun and V. Clarke, "Using thematic analysis in psychology," *Qualitative Research in Psychology*, vol. 3, no. 2, pp. 77–101, 2006.